

Version Control Workflow Guide

How the team will use Git and GitHub to collaborate on the AI-Powered Code Review System project: forking, branching, committing, raising pull requests, and merging.

1. Roles

Repo Owner (Yashwanth): owns the main repository, reviews pull requests, merges approved work.

Contributors (Srija & Akhil): work in their own forks, submit pull requests for review.

2. One-Time Setup (each contributor(Srija & Akhil) does this once)

2.1 Fork the repository

On GitHub, open the main repository and click the "Fork" button. This creates a personal copy of the repo under your own GitHub account.

2.2 Clone your fork to your computer

```
git clone https://github.com/<Your-Username>/AI-Powered-Code-Review-System
cd AI-Powered-Code-Review-System
```

3. Creating a Branch for Each Feature

Never work directly on the main branch. Create a new branch for each feature or task — this keeps your changes isolated and makes pull requests clean and easy to review. Every time you create a branch make sure

you click sync in the github so you are always synced with the main repo

```
git switch main
git pull origin main
git switch -c feature/short-descriptive-name ->this will create a branch and switch
```

Examples of good branch names:

- feature/dashboard-charts
- feature/oauth-repo-connect
- feature/settings-page

If you're fixing a bug rather than adding a feature, prefix with fix/ instead of feature/.

4. Making and Saving Changes

Work normally in your code editor. Once you've made progress worth saving:

```
git add .
git commit -m "Add chart for issue trends over time"
```

Write commit messages that describe what changed, in plain language. Commit often — small, clear commits are easier to review than one giant commit at the end.

5. Pushing Your Branch

Push your branch to your own fork (not the main repo — you don't have write access there, which is intentional):

```
git push origin branch_name
```

The first time you push a new branch, GitHub will show a link in your terminal to open a pull request directly. You can also do this from the GitHub website.

6. Raising a Pull Request (PR)

Go to your fork on GitHub. You'll see a banner: "Compare & pull request" — click it.

Make sure the base repository is the owners' repo's main branch, and the head is your feature branch.

Write a clear title and description: what you built, what to test, anything the reviewer should know.

Click "Create pull request."

This does not merge anything automatically. It only opens a request for the repo owner to review.

7. Reviewing and Merging (Repo Owner's job)

Open the PR on GitHub, review the "Files changed" tab.

Leave comments on specific lines if something needs fixing — the contributor can push more commits to the same branch and they'll appear in the same PR automatically.

Once satisfied, click "Merge pull request," then "Confirm merge."

Delete the branch after merging (GitHub will prompt you) to keep things tidy.

8. Keeping Your Fork Up to Date

After the repo owner merges a PR (yours or someone else's), everyone should sync their fork's main branch before starting new work: click sync fork on the github

Do this before creating any new feature branch, so you're always building on top of the latest merged code.

9. Handling Merge Conflicts (yashwanth's task)

A conflict happens when two people changed the same lines in the same file. Git will tell you which files have conflicts when you try to merge or pull.

Open the conflicted file — Git marks the conflicting sections with <<<<<<, =====, and >>>>>> markers.

Manually edit the file to keep the correct/combined version, then remove the marker lines.

Save the file, then run:

```
git add <filename>
git commit
```

This completes the merge with your conflict resolution included. If you're unsure which version is correct, ask the other person before resolving.

10. Quick Command Reference

Action	Command
Fork the repo	Use the GitHub website "Fork" button
Clone your fork	<code>git clone https://github.com/<your-username>/<repo>.git</code>
Add upstream remote	<code>git remote add upstream https://github.com/<owner>/<repo>.git</code>
Create a branch	<code>git checkout -b feature/dashboard-charts</code>
Check current branch	<code>git branch</code>
Stage changes	<code>git add .</code>
Commit changes	<code>git commit -m "Add scan history chart"</code>
Push branch to your fork	<code>git push origin feature/dashboard-charts</code>
Sync fork with upstream	<code>git fetch upstream && git checkout main && git merge upstream/main</code>
Update your branch with latest main	<code>git checkout feature/dashboard-charts && git merge main</code>
Delete a merged local branch	<code>git branch -d feature/dashboard-charts</code>

11. Ground Rules for This Project

Never push directly to the main branch of the main repo — always go through a pull request.

Pull the latest main into your branch before opening a PR, so conflicts are caught early, not at merge time.

One feature per branch and per PR — don't bundle unrelated changes together.

Keep commit messages descriptive — "fix stuff" helps no one during review or in the report writeup later.

Sync your fork daily if others are merging frequently, to avoid falling far behind main.